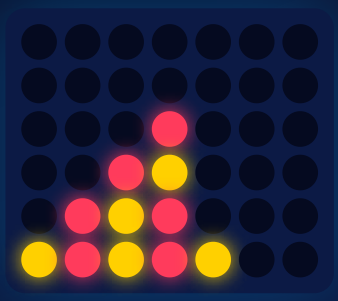
JAVA DERSİ • TEKNİK PROJE RAPORU

CONNECT 4 GUI



JavaFX tabanlı; yapay zekâya karşı, yerel iki kişilik ve iki bilgisayar arası ağ oyunu destekleyen masaüstü Connect 4 uygulaması.

Frontend — **Mustafa** • Backend — **Nawar** • AI & Ağ — **Ruslan**

24 sınıf • ~3.980 satır • 6 paket • JavaFX 17+ • Sıfır dış bağımlılık

İçindekiler

1. Yönetici Özeti
2. Teknoloji ve Gereksinimler
3. Katmanlı Mimari (UML Paket Diyagramı)
4. Genel Sınıf Diyagramı
5. core.constants — Yapılandırma
6. core.logic — Oyun Motoru & AI
7. core.model — Profil & Ödüller
8. core.network — Ağ Katmanı
9. ui — Kullanıcı Arayüzü
10. util — Yardımcılar
11. Ağ Protokolü (Sıra Diyagramı)
12. Yapay Zekâ Algoritması
13. Derleme ve Çalıştırma
14. Sonuç

1 Yönetici Özeti

Connect 4 GUI, klasik "dört taşı dizen kazanır" oyununu modern bir masaüstü uygulamasına dönüştürür. Proje üç ana hedefi karşılar:

- **Oynanabilirlik:** Yerel iki kişilik mod, dört zorluk seviyeli yapay zekâ ve iki bilgisayar arası ağ oyunu.
- **Kalıcılık & ilerleme:** XP/seviye sistemi, galibiyet serileri, 10 başarım ve maç geçmişi diske kaydedilir.
- **Kullanıcı deneyimi:** Sinematik animasyonlar, beş renk teması, açık/koyu mod ve çalışma anında üretilen ses efektleri.

Uygulama **katmanlı bir mimari** ile tasarlanmıştır: arayüz (`ui`), oyun mantığı (`core.logic`), veri modeli (`core.model`), ağ (`core.network`) ve ortak sabitler (`core.constants`) birbirinden net şekilde ayrılır. Bağımlılıklar tek yönlüdür: arayüz mantığa bağımlıdır, mantık arayüzden habersizdir. Bu sayede oyun motoru ve AI, arayüzden bağımsız olarak test edilebilir ve ağ oyununda yeniden kullanılır.

> **Mimari ilkesi:** Hem `NetworkHost` hem `NetworkClient` , aynı `NetworkPeer` arayüzünü uygular. Böylece `GameScreen` karşısındakinin sunucu mu istemci mi olduğunu bilmeden tek bir kod yoluyla çalışır — bu, *polimorfizm*in projedeki en güçlü kullanımıdır.

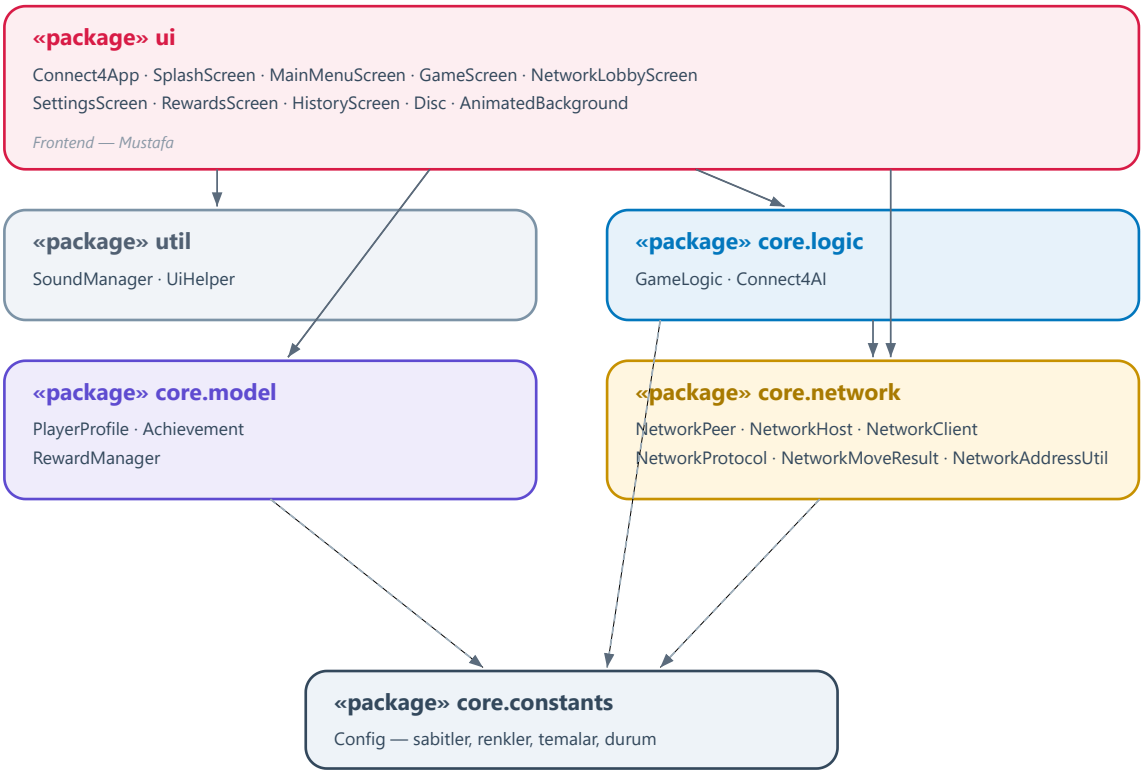
2 Teknoloji ve Gereksinimler

Bileşen	Kullanım
Java 17+	switch ifadeleri, `record` , metin blokları gibi modern dil özellikleri.
JavaFX Controls	Label, TextField, ScrollPane gibi arayüz bileşenleri.
JavaFX Graphics + Canvas	Animasyonlu arka plan, geçişler, özel çizimler.
JavaFX Media (AudioClip)	Ses efektlerinin çalınması.
java.net (TCP Socket)	İki bilgisayar arası ağ oyunu, port 5555.
java.nio.file	Oyuncu profilinin düz metin dosyasına kaydı.
JPMS (module-info)	Modüler proje yapısı, JavaFX modüllerine erişim.

> **Öne çıkan tercih:** Proje hiçbir dış kütüphane kullanmaz. Ses efektleri bile harici WAV dosyası yerine, çalışma anında sinüs dalgalarından matematiksel olarak üretilir (`SoundManager`). Arka plan animasyonları video değil, saf Canvas çizimidir.

3 Katmanlı Mimari — UML Paket Diyagramı

Oklar **bağımlılık yönünü** gösterir (A → B: "A, B'yi kullanır"). Tüm bağımlılıklar yukarıdan aşağıya akar; alt katmanlar üst katmanlardan habersizdir.

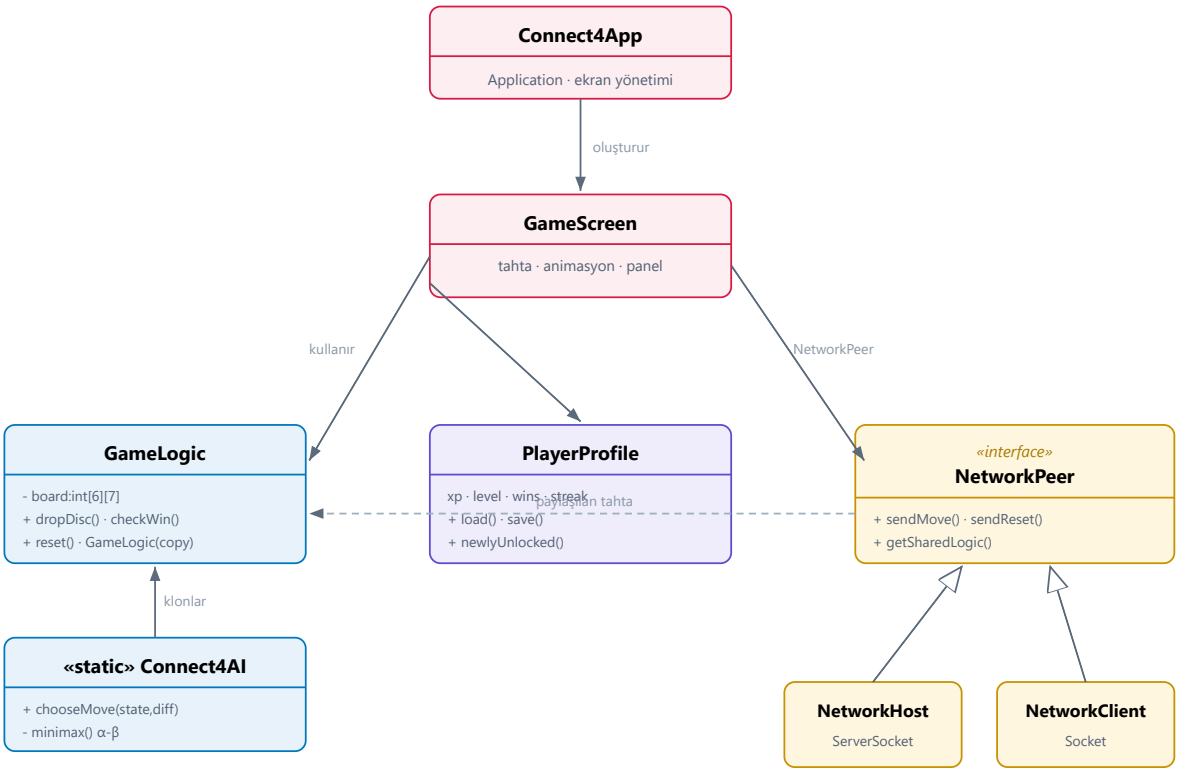


Şekil 1 — Paketler arası bağımlılıklar. Kesikli oklar: herkesin ortak `Config`'e bağımlılığı.

Sunum (UI) Mantık Model Ağ Yardımcı

4 Genel Sınıf Diyagramı (Çekirdek İlişkiler)

Aşağıdaki UML, en kritik sınıflar arasındaki ilişkileri özetler: **kalıtım/uygulama** (içi boş üçgen), **birleşim/kullanım** (düz ok).



Şekil 2 — Çekirdek sınıf ilişkileri. İçi boş üçgen = arayüz uygulama (implements).

5 core.constants — Yapılandırma Katmanı

Config `final · util`

Tüm projenin tek "doğruluk kaynağı": boyutlar, renkler, temalar, oyun sabitleri ve global çalışma-zamanı durumu.

Üye / Metot	Görevi
`ROWS, COLS, WIN_LENGTH`	6×7 tahta ve 4'lü kazanma kuralı.
`PLAYER_1, PLAYER_2, EMPTY`	Hücre durum sabitleri.
`COLOR_* (Color)`	Aktif tema renkleri; tema/mod değişince güncellenir.
`applyTheme(int)`	5 renk temasından birini yükler (oyuncu/tahta/parıltı renkleri).
`applyDisplayMode(boolean)`	Açık/koyu mod paletini uygular.
`panelFill / cardFill / modalBg…`	Moda duyarlı yardımcı renk üreticileri.
`scorePlayer1, soundEnabled…`	Oturum boyu global durum (skor, ses, seçili tema).
`NETWORK_PORT = 5555`	Ağ oyununun TCP portu.

Tasarım notu: Private kurucu + `final` sınıf ile örneklenemez; statik bir yapılandırma havuzu olarak çalışır.

6 core.logic — Oyun Motoru ve Yapay Zekâ

GameLogic `çekirdek`

Tahtanın durumunu ve oyunun kurallarını tutan, arayüzden tamamen bağımsız saf model. Test edilebilir ve yeniden kullanılabilir.

Metot	Görevi
`dropDisc(col)`	Sütuna taş bırakır (yerçekimi), kazanma/beraberlik kontrol eder, sırayı çevirir; düşen satırı döndürür.
`checkWin(row,col)`	Son taştan 4 yöne (yatay, dikey, 2 çapraz) bakarak 4'lü dizi arar; bulursa kazanan hücreleri saklar.
`getPlayableColumns()`	Oynanabilir sütun listesi — AI bunu kullanır.
`GameLogic(GameLogic)`	**Kopya kurucu** — AI'nin tahtayı bozmadan geleceği simüle etmesini sağlar.
`isDraw / getWinner / reset`	Beraberlik tespiti, kazanan sorgusu, yeni maç için sıfırlama.

Connect4AI `final · static`

Zorluk seviyesine göre yapay zekânın hamlesini seçen, durumsuz yardımcı sınıf.

Seviye / Metot	Strateji
`EASY → randomMove`	Rastgele geçerli sütun.
`MEDIUM → mediumMove`	Kazanabiliyorsa kazanır, rakip kazanacaksa engeller, yoksa ortayı seçer.
`HARD → bestMove(5)`	5 derinlikli minimax.
`EXPERT → bestMove(7)`	7 derinlikli minimax.
`minimax(α,β)`	Alfa-beta budamalı özyinelemeli arama; kazanç +1.000.000, kayıp −1.000.000.
`evaluate / countThreats`	Yaprak düğümlerde taş dizilerini (tehditleri) sayan sezgisel skor.

7 core.model — Profil ve Ödül Sistemi

PlayerProfile `kalıcı veri`

Oyuncunun tüm ilerlemesini tutan ve diske kaydeden model. `~/.connect4gui_profile.txt` *dosyasında* `anahtar=değer` *formatında saklanır.*

Metot	Görevi
`load() / save()`	Profili düz metin dosyasından okur / dosyaya yazar.
`addXp / recalculateLevel`	XP ekler; artan eşiklerle (200, +80…) seviyeyi yeniden hesaplar.
`recordWin/Loss/Draw`	Galibiyet, kayıp, beraberlik istatistiklerini ve seriyi günceller.
`newlyUnlocked(all)`	Yeni açılan başarımları döndürür ve XP ödülünü ekler.
`addHistory(entry)`	Tarih damgalı maç kaydı ekler (son 30 maç tutulur).

Achievement `değişmez`

Tek bir başarımın değişmez (immutable) tanımı: kimlik, başlık, açıklama, ikon, gerekli seviye ve XP ödülü.

RewardManager `final · static`

10 başarımın merkezi kayıt defteri ve zorluk başına XP ödül tablosunu sağlayan yardımcı.

Metot	Görevi
`getAllAchievements()`	Tüm başarım tanımlarını döndürür.
`xpForWin/Draw/Loss(diff)`	Sonuca ve zorluğa göre kazanılan XP'yi belirler (galibiyet 60→250).

8 core.network — Ağ Katmanı

İki bilgisayar arası oyun, **otoriter sunucu** modeliyle çalışır: sunucu oyunun tek doğru kopyasını tutar, istemci yalnızca aynalar. Bu, tutarsızlığı ve hileyi önler.

NetworkPeer «interface»

Host ve Client'ın ortak sözleşmesi. `GameScreen` *bu arayüz üzerinden çalışır, karşısındakinin türünü bilmez (polimorfizm).*

sendMove · sendReset · setMoveListener · setDisconnectListener · setErrorListener · disconnect · isHost · getLocalPlayer · getSharedLogic

NetworkHost sunucu

Port 5555'te `ServerSocket` *açar, istemci bağlantısını kabul eder ve oyunun otoriter* `GameLogic` *örneğini yönetir.*

- **acceptLoop:** istemciyi bekler, JOIN'i okur, WELCOME gönderir.
- **processMove (synchronized):** sıra ve geçerlilik kontrolü → taşı bırakır → her iki tarafa RESULT yayınlar.
- Daemon thread havuzu ile arayüzü bloklamaz.

NetworkClient istemci

Sunucuya bağlanır, hamleleri gönderir ve gelen RESULT'ları yerel ayna tahtaya uygulayarak arayüzü günceller. Reset başlatamaz.

İç `record WelcomeInfo(player, hostName, clientName)` ile karşılama bilgisini taşır.

NetworkProtocol final · static

Satır tabanlı metin protokolünün kodlayıcı/çözücüsü. Mesajları `|` *ile ayırır, girdileri temizler (sanitize).*

NetworkMoveResult DTO

Bir hamlenin sonucunu (sütun, satır, oyuncu, sıra, oyun bitti mi, kazanan, kazanan hücreler) taşıyan değişmez veri nesnesi.

NetworkAddressUtil final · static

Yerel ağ IP adreslerini keşfeder (loopback/sanal arayüzleri eler) — kullanıcının rakibe vereceği IP'yi bulur.

9 ui — Kullanıcı Arayüzü

Sınıf	Görevi
`Connect4App`	JavaFX `Application` giriş noktası; sahneyi kurar, ekranlar arası geçişi ve profil/ağ yaşam döngüsünü yönetir.
`SplashScreen`	Açılış ekranı; parıldayan logo, otomatik veya tıklamayla menüye geçiş.
`MainMenuScreen`	İsim girişi, zorluk/mod seçimi, tema değiştirme ve diğer ekranlara yönlendirme.
`GameScreen`	Projenin en geniş ekranı: tahta çizimi, taş düşme animasyonu, geri sayım, yan istatistik paneli, AI/ağ akışı, sonuç ekranı ve konfeti.
`NetworkLobbyScreen`	Oda kurma (sunucu) ve odaya katılma (istemci) arayüzü; IP/durum geri bildirimi.
`SettingsScreen`	Ses efekti açma/kapama ve 5 renk teması seçimi.
`RewardsScreen`	Başarım kartları, seviye ilerleme çubuğu, XP durumu.
`HistoryScreen`	Kaydedilen son maçların listesi.
`Disc`	Tek bir taşı temsil eden `Circle`; renk/parıltı ve kazanma nabız animasyonu.
`AnimatedBackground`	Canvas tabanlı canlı arka plan: nefes alan gradyan, bokeh küreler, düşen diskler, parçacık takımyıldızı ve vinyet.

10 util — Yardımcılar

SoundManager `final · static`

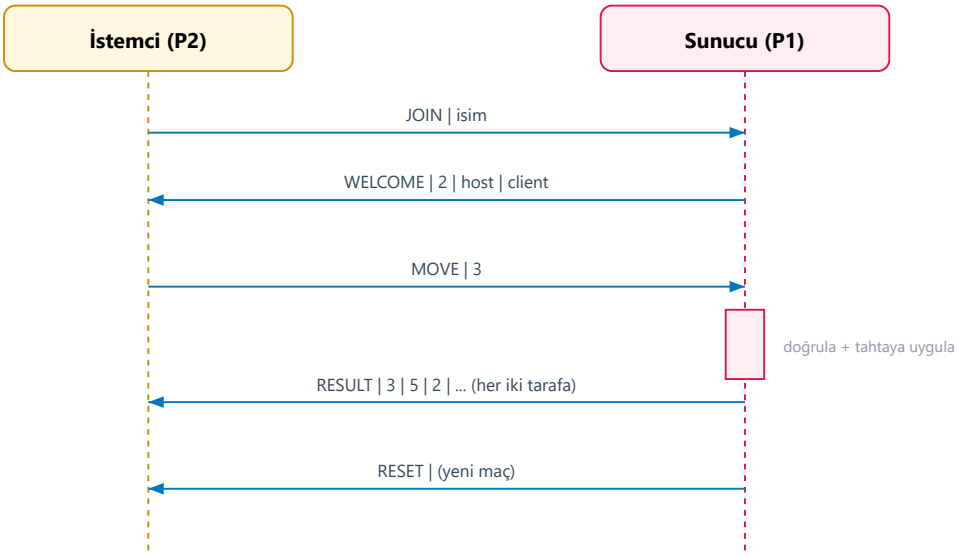
8 ses efektini (damla, zafer, kayıp, geri sayım…) çalışma anında sinüs dalgalarından WAV olarak üretir ve `AudioClip` ile çalar — harici ses dosyası gerektirmez.

UiHelper `final · static`

Tekrar eden arayüz öğelerinin fabrikası: başlık etiketi, menü butonu, geri bağlantısı, duyarlı ölçek hesabı, parıltı animasyonu. Ekranlar arası tutarlılık sağlar.

11 Ağ Protokolü — Sıra Diyagramı

Mesaj	Yön	Anlamı
`JOIN|isim`	İstemci → Sunucu	Bağlanma ve isim bildirimi.
`WELCOME|2|host|client`	Sunucu → İstemci	Karşılama; istemciye oyuncu no atanır.
`MOVE|sütun`	İstemci → Sunucu	Hamle isteği.
`RESULT|col|row|...`	Sunucu → Her iki taraf	Onaylanmış hamle sonucu (otoriter).
`RESET|`	Sunucu → İstemci	Yeni maç (yalnız sunucu başlatır).
`ERROR|mesaj`	Sunucu → İstemci	Geçersiz/sıra dışı hamle uyarısı.



Şekil 3 — Tipik bir ağ oturumunun mesaj akışı.

12 Yapay Zekâ Algoritması

Zor ve Uzman seviyelerde AI, **minimax** kullanır: oyun ağacını verilen derinliğe kadar açar, kendi sırasında skoru *en üst düzeye*, rakip sırasında *en alt düzeye* çıkaracağını varsayar. **Alfa-beta budama**, sonucu değiştirmeyecek dalları erken keserek aramayı hızlandırır.

> **Skorlama:** Kazanma `+1.000.000 + derinlik`, kaybetme `-1.000.000 - derinlik` olarak puanlanır. "+derinlik" terimi, AI'nin *en hızlı* galibiyeti ve *en geç* yenilgiyi tercih etmesini sağlar. Belirsiz konumlarda `evaluate()` sezgiseli, taraların oluşturduğu 2'li/3'lü tehditleri sayarak konumu puanlar.

Karar akışı

1. Oynanabilir her sütun için tahta klonlanır ve taş denenir.
2. Sonuç durumu `minimax`'e verilir (bir alt seviye, sıra rakipte).
3. En yüksek skoru veren sütun seçilir.
4. Orta seviye bu ağacı kullanmaz; tek hamlelik kazanç/engelleme sezgisiyle çalışır.

13 Derleme ve Çalıştırma

Gereksinimler

Java 17+ ve JavaFX SDK (önerilen: `javafx-sdk-26.0.1`).

Komut satırı (PowerShell)

```
$JFX = "C:\...\javafx-sdk-26.0.1\lib"
javac --module-path $JFX --add-modules javafx.controls,javafx.media,javafx.graphics `
  -d bin -sourcepath src src/module-info.java src/core/**/*.java src/util/*.java src/ui/*.java
java --module-path "$JFX;bin" --add-modules javafx.controls,javafx.media,javafx.graphics `
  --module Connect4GUI/ui.Connect4App
```

İki bilgisayardan oynama

1. Her iki PC aynı Wi-Fi/LAN ağında olmalı.
2. Sunucu: Ana menü → **Ağ Oyunu** → **Oda Kur** → görünen IP'yi paylaş.
3. İstemci: aynı menü → IP'yi gir → **Odaya Katıl** → oyun otomatik başlar.
4. Gerekirse Windows Güvenlik Duvarı'nda **port 5555** için Java'ya izin verin.

14 Sonuç

Connect 4 GUI; nesne yönelimli tasarım, katmanlı mimari, algoritma (minimax), ağ programlama (TCP soket), kalıcı veri yönetimi ve zengin bir arayüz geliştirmeyi tek projede birleştirir. Sorumlulukların net ayrımı sayesinde oyun motoru hem yerel hem ağ modunda yeniden kullanılır; `NetworkPeer` soyutlaması ile arayüz, ağın iki ucunu da tek tip ele alır.

> **Kazanımlar:** arayüz/iş mantığı ayrımı, arayüz (interface) ve polimorfizm, özyinelemeli arama + budama, eşzamanlılık (daemon thread'ler), dosya G/Ç ve olay tabanlı programlama bu projede uçtan uca uygulanmıştır.